Data structures and algorithms using C++

LU 1: Apply c++ programming concepts

Resource folder: https://shorturl.at/psDW7

Purpose statement of the Course

At the end of this module you will be able to apply data structures in problem-solving scenarios and adeptly write C++ programs.You will learn to Apply C++ programming concepts, Analyze data structure and algorithms, Implement Linear Data Structure, Implement Non Linear Data Structure, Implement Hash Table and handle Data Retrieval, and Apply recursion for solving complex problems using C++.

Additionally, you will gain a comprehensive understanding of hash table implementations for efficient data retrieval and collision handling. With these competences, you will be empowered to tackle complex challenges independently, ensuring efficient and effective problem-solving in C++ programming.

Definition of Key terms

Data Structure:

Data Structure is a logical or mathematical model for collecting and organizing data in such a way that we can perform operations on these data in an effective way. Data Structures is about rendering data elements in terms of some relationship, for better organization and Storage.

Algorithm

An algorithm is **a set of instructions for solving a problem or accomplishing a task.**

It can also be defined as a process or set of rules to be followed in calculations or other problem-solving operations, especially by a computer.

OVERVIEW OF C++ PROGRAMMING

What is c++?

C++ is a cross-platform language that can be used to create high-performance applications.

C++ was developed by Bjarne Stroustrup at Bell labs in 1979, as an extension to the C language.

C++ gives programmers a high level of control over system resources and memory.

The language was updated 3 major times in 2011, 2014, and 2017 to C++11, C++14, and C++17.

Why USE C++?

C++ is one of the world's most popular programming languages.

C++ can be found in today's operating systems, Graphical User Interfaces, and embedded systems.

C++ is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs.

C++ is portable and can be used to develop applications that can be adapted to multiple platforms.

C++ is fun and easy to learn!

Evolution of C++

Evolution of C++ can be traced back to 1980 when Bjarne Stroustrup developed a language he referred to as “C with Classes” at Bell Laboratories. Motivated object-oriented programming pioneered in Smalltalk, Stroustrup included powerful features into C with design goal of supporting object-oriented programming while retaining backward compatibility with C.

By 1984, more enhancements had been added to “C with Classes” hence it was renamed C++. Therefore, the name C++ uses C increment operator (++) to indicate that

C++ is an enhancement of C. This integration of object orientation into procedural-oriented C makes C++ a multiparadigm language suitable for developing system software like operating systems.

Features of C++

The following are general features of C++ that makes it one of the most powerful and flexible programming supported by most computers.

1. **Portability:** Programs written in C++ are portable across multiple hardware and software platforms.
2. **Object-oriented programming:** The design goal of C++ is to support object- oriented programming. As mentioned earlier, instead of using function that access global variables, both data and variables are encapsulated into an object.
3. **Keywords:** Keywords also referred to as **reserved words** are words that have special meaning in a language. C++has a large number such as include, main, while, for, if, else and return.
4. **Operators:** Operators are used to evaluate an expression that returns a value. This will be discussed in details later.
5. **Storage in memory:** In C++ a variable is a named storage location in computer memory for holding data of a particular type.
6. **Identifiers:** In C++ programming, identifiers are symbolic names used to identify elements like variables and constants in a program. Because C++ s case sensitive, it is important to observe caution when creating user-defined identifiers.
7. **Case sensitive:** C++ is case-sensitive. Myname and MyName are different identifiers
8. **Type checking:** C++ provides a rules and mechanism for checking data types before execution starts. If a compiler detects inconsistence, it ensures that the data conversions defined in the language or by the user do not cause runtime errors or system failure

Environment

C++ Get Started

To start using C++, you need two things:

- A text editor, like Notepad to write C++ code
- A compiler, like GCC, to translate the C++ code into a language that the computer will understand

There are many text editors and compilers to choose from. In this tutorial, we will use an IDE (see below).

C++ Install IDE

An IDE (Integrated Development Environment) is used to edit AND compile the code.

Popular IDE's include Code::Blocks, Eclipse,sublime text and Visual Studio. These are all free, and they can be used to both edit and debug C++ code.

Setting environment

Compiler

If you are using Visual Studio, then you're done! Visual Studio comes with Microsoft's Visual C++ compiler, so no further steps are necessary. If not, then you'll need a separate C++ compiler.

Compiler:GNU Compiler Collection.

GCC is the GNU Compiler Collection. It provides compiler front-ends for several languages, including C, C++, Objective-C, Fortran, Ada, and Go. It also includes runtime support libraries for these languages.

GCC provides many levels of source code error checking traditionally provided by other tools (such as lint), produces debugging information, and can perform many different optimizations to the resulting object code.

1 Install GCC compiler

```
sudo apt-get install build-essential
```

2. Install the debugger

```
sudo apt-get install gdb
```

Windows users(install MinGW):
https://www.scaler.com/topics/c/c-compiler-for-windows/

Installation guide

Windows users

https://shorturl.at/I7Z5D

LInux Users

1 Install GCC compiler

```
sudo apt-get install build-essential
```

2**. Install the debugger**

```
sudo apt-get install gdb
```

First program

Step 1. Now go to that folder where you will create C++ programs. Editors can provide alternative features for compiling and running the program. Below, I am creating my programs in Documents directory. Type these commands(Linux Users):

$ cd Documents/
$ sudo mkdir programs
$ cd programs/

Step 2. Open a file using any editor.
$ sudo gedit hello.cpp (for C++ programs)

Step 3. Add the code(from right) in the file save the file and exit

Step 4. Compile the program using any of the following command
$ sudo g++ -o hello hello.cpp

Step 5. To run this program type this command
$ ./hello

```
#include <iostream>
using namespace std;
int main()
{
    cout << "Hello World! ";

    return 0;
}
```

Note: A C program is a C++ program

```
#include <stdio.h>
int main()
{
    printf( "Hello World! ");
    return 0;
}
```

When saved as hello.cpp, the program will compile

Example 2: Every C Program is a C++ Program

```
#include <stdio.h>
int main()
{
int a,b,sum;
printf("Enter a and b:\n");
scanf("%d %d",&a,&b);
sum=a+b;
printf("The sum of  %d + %d = %d\n",a,b,sum);
return 0;
}
```

Save, Compile and run the C program as a C++ program under name addition.cpp

```
damascene10@damascene10-Latitude-3400:~$ g++ -o addition addition2.cpp
damascene10@damascene10-Latitude-3400:~$ ./addition
Enter a and b:
12 13
The sum of 12+13=25
damascene10@damascene10-Latitude-3400:~$
```

Example 2(cont'd) : Addition.cpp rewritten using C++ features not available in C

```
#include <iostream>
using namespace std;
int main()
{
int a,b,sum;
cout<<"Enter a and b:\n";
cin>>a>>b;
sum=a+b;
cout<<"The sum of "<< a<<"+"<<b<<"="<<sum<<endl;
return 0;
}
```

The previous program is Written using alternative C++ features

Note the use of iostream, namespacem cin and cout

Note: Don't worry if you don't understand the details, everything will be explained shortly

```
damascene10@damascene10-Latitude-3400:~$ g++ -o addition addition2.cpp
damascene10@damascene10-Latitude-3400:~$ ./addition
Enter a and b:
12 13
The sum of 12+13=25
damascene10@damascene10-Latitude-3400:~$
```

C++ Syntax

In C++, a program may consist of objects, functions, variables, and other components. However, regardless of size or complexity of a C++ program, the program has to include directives, and at least one function called main. The following is the basic structure of C++ program:

```
#include directive;

global variables/constants;

user_defined_functions

return_type main(){

executable statements //comment

return something

}
```

```
#include <iostream>
using namespace std;
int main()
{
    cout << "Hello World! ";

    return 0;
}
```

Example program explained

Line 1: `#include <iostream>` is a header file library that lets us work with input and output objects, such as `cout` (used in line 5). Header files add functionality to C++ programs.

Line 2: `using namespace std` means that we can use names for objects and variables from the standard library. Using namespace std; namespace is a feature in C++ used to ensure that identifiers do not overlap due to naming conflict. Identifiers may overlap by sharing different parts of a program. Each namespace such as std (standard) defines a scope in which identifiers are placed.

Don't worry if you don't understand how `using namespace std` works. Just think of it as something that (almost) always appears in your program.

Line 3: Another thing that always appear in a C++ program, is `int main()`. This is called a function. Any code inside its curly brackets `{}` will be executed.

Line 4: `cout: Console output` (pronounced "see-out") is an object used to output/print text. In our example it will output "Hello World".

Note: Every C++ statement ends with a semicolon `;`.

Note: The body of `int main()` could also been written as:

```
int main () { cout << "Hello World! "; return 0; }
```

Remember: The compiler ignores white spaces. However, multiple lines makes the code more readable.

Line 5: `return 0` ends the main function.

C++ Output (Output stream)

Output capabilities in C++ are provided by a library file known as ostream. The ostream has an object called cout that stands for console out that prints output on a standard output, usually a display screen. The object is used in conjunction with stream insertion operator, which is written as << (two “less than” signs).

The `cout` object, together with the `<<` operator, is used to output values/print text:

`cout` is pronounced "see-out". Used for output, and uses the **insertion operator (<<)**

You can add as many `cout` objects as you want. However, note that it does not insert a new line at the end of the output:

```
#include <iostream>
using namespace std;
int main() {
  cout << "Hello World!";
  cout << "I am learning C++";
  return 0;
}
```

New Lines

To insert a new line, you can use the `\n` character. Two `\n` characters after each other will create a blank line:

```
#include <iostream>

using namespace std;
int main() {
  cout << "Hello World! \n";//New line
  cout << "Hello World again! \n\n";// Blank
line
  cout << "I am learning C++";
  return 0;
}
```

Another way to insert a new line, is with the `endl` manipulator:

```
#include <iostream>
using namespace std;
int main() {
  cout << "Hello World!" << endl;
  cout << "I am learning C++";
  return 0;
```

C++ input stream(User input)

In C++ handling input is done using the cin combined with **>>** known as stream
extraction operator. cin represents the standard input device (or **C**onsole **IN**put),
i.e., keyboard. The symbol >> after the cin. The >> operator causes data input such
as an integer value to be input from cin into computer memory.

You have already learned that `cout` is used to output (print) values. Now we will use `cin` to get user input.

`cin`:Console input is a predefined variable that reads data from the keyboard with the **extraction operator** (`>>`).

In the following example, the user can input a number, which is stored in the variable `x`. Then we print the value of `x`:

Example

```
int x, y;
int sum;
cout << "Type a number: ";
cin >> x;
cout << "Type another number: ";
cin >> y;
sum = x + y;
cout << "Sum is: " << sum;
```

Data types

The computer memory is organised into cells that can store one or more bytes. A byte is the minimum amount of memory that we can manage in C++. To declare a variable, you must declare the type of variable so that the computer reserves enough bytes to store a value of that type.

Primitive types:

int, float, double, boolean and char

Complex data types

A complex data type is a combination data of similar or different types. C++ supports complex data types such as string, array, struct (record), enumerated type, linked lists, and pointers. Apart from string data type, other examples of complex data types include arrays, linked lists, stacks, queues, trees and graphs.

C++ Strings

Strings are used for storing text.

A `string` variable contains a collection of characters surrounded by double quotes: eg. `string greeting = "Hello";`

To use strings, you must include an additional header file in the source code, the `<string>` library:

String Concatenation

The `+` operator can be used between strings to add them together to make a new string. This is called concatenation:

Example

```
string firstName = "John ";
string lastName = "Doe";
string fullName = firstName + lastName;
cout << fullName;
```

String length

A string in C++ is actually an object, which contain functions that can perform certain operations on strings. For example, the length of a string can be found with the `length()` function:

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
cout << "The length of the txt string is: " <<
txt.length()
```

Access Strings

You can access the characters in a string by referring to its index number inside square brackets `[]`.

This example prints the first character in myString:

```
string myString = "Hello";

cout << myString[0];
// Outputs H
```

Change String Characters

To change the value of a specific character in a string, refer to the index number, and use single quotes:

Example

```
string myString = "Hello";

myString[0] = 'J';

cout << myString;

// Outputs Jello instead of
Hello
```

User input string

User Input Strings

It is possible to use the extraction operator `>>` on `cin` to display a string entered by a user:

Example

```
string firstName;
cout << "Type your first name: ";
cin >> firstName; // get user input from the keyboard
cout << "Your name is: " << firstName;

// Type your first name: John
// Your name is: John
```

However, `cin` considers a space (whitespace, tabs, etc) as a terminating character, which means that it can only display a single word (even if you type many words):

```
string fullName;
cout << "Type your full name: ";
cin >> fullName;
cout << "Your name is: " << fullName;

// Type your full name: John Doe
// Your name is: John
```

From the example above, you would expect the program to print "John Doe", but it only prints "John".

Exercises

1. Write a program that prompt the user to enter the firstName,lastName,age and display all information in one line.
2. Write a program which asks the user to enter
    - fullname
    - initial amount,
    - the interest rate per year,
    - payment time in terms of years

And finally Compute and display the total

getline

That's why, when working with strings, we often use the `getline()` function to read a line of text. It takes `cin` as the first parameter, and the string variable as second:

```
string fullName;

cout << "Type your full name: ";

getline (cin, fullName);

cout << "Your name is: " << fullName;
// Type your full name: John Doe
// Your name is: John Doe
```

Note: When using getline() after some inputs, the function will fail to read inputs because when cin.getline() reads from the input, there is a newline character left in the input stream, so it doesn't read your c-string. Use cin.ignore() before calling getline().

```
cout<<"Enter age";
cin>>age;
cout<<"Enter fullname:\t";
cin.ignore();
getline(cin,fullname);
```

C++ IDENTIFIERS

The names of variables, functions, labels, and various other user-defined **objects are called identifiers.**

All C++ variables must be identified with unique names.

These unique names are called identifiers.

Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

Note: It is recommended to use descriptive names in order to create understandable and maintainable code.

The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore (_)
- Names are case sensitive (`myVar` and `myvar` are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (like C++ keywords, such as `int`) cannot be used as names

Keywords: Reserved words

C++ has reserved words, which have a unique meaning and must not be used for any other purposes. The reserved words already used are **main, int , return**, and **using**. See other examples below:

Else, new,this, auto, enum, bool, true, private, try, public, protected, case, false, true, break, class, for, return,const, signed, unsigned, void, static, int, float, continue, if, break, switch, while, etc…..

Namespace

Consider following C++ program.

```
// A program to demonstrate need
of namespace

int main()

{

  int value;

  value = 0;

  double value; // Error here

  value = 0.0;

}
```

Output :

```
Compiler Error:

'value' has a previous declaration

as 'int value'
```

In each scope, a name can only represent one entity. So, there cannot be two variables with the same name in the same scope. Using namespaces, we can create two variables or member functions having the

Namespaces:**Definition and Creation:**

Namespaces allow us to group named entities that otherwise would have *global scope* into narrower (limited) scopes, giving them *namespace scope*. This allows organizing the elements of programs into different logical scopes referred to by names.

- Namespace is a feature added in C++ and not present in C.
- A namespace is a declarative region that provides a scope to the identifiers (names of the types, function, variables etc) inside it.
- Multiple namespace blocks with the same name are allowed. All declarations within those blocks are declared in the named scope.

A namespace definition begins with the keyword **namespace** followed by the namespace name as follows:

```
namespace namespace_name
{
  int x, y; // code declarations where x and y are declared in
namespace name's scope
```

Namespace declarations appear only at global scope.

- Namespace declarations can be nested within another namespace.
- Namespace declarations don’t have access specifiers. (Public or private)
- No need to give semicolon after the closing brace of definition of namespace.
- We can split the definition of namespace over several units.

Example program with Multiple namespaces

```
// Creating namespaces
#include <iostream>
using namespace std;
namespace namesp1
{
        int value() { return 5; }
}
namespace namesp2
{
        const double x = 100;
        double value() { return 2*x; }
}
int main()
{
        // Access value function within namesp1
        cout << namesp1::value() << '\n';

        // Access value function within namesp2
        cout << nsmesp2::value() << '\n';

        // Access variable x directly
        cout << namesp2::x << '\n';
        return 0;
}
```

Namespace : Consider the following program

```
/* Here we can see that more than one
variables are being used without reporting
any error.

That is because they are declared in the
different namespaces and scopes.*/

#include <iostream>

using namespace std;
// Variable created inside namespace

namespace first
{
        int amount= 5000; }

// Global variable

int amount= 2000;
```

```
int main()

{

        // Local variable

        int amount = 3000;

/*These variables can be accessed from outside
the namespace using the scope operator ::*/

cout << first::amount << '\n'; //Access variable
amount created inside  first namespace

cout << ::amount << '\n'; //Access variable
amount as a global variable

cout << amount << '\n'; //Access  local variable
amount;

return 0;

}
```

Exercice

1. Create a program with a namespace called userDefined with a variable insideNamespace of type integer and give it initial value.
   a. Create a function cout inside the namespace(userDefined) which return integer as the value of insideNamespace.
   b. Create a global variable myGlobal of type integer and give it initial value
   c. Create a global function cout and return the square of myGlobal
   d. Create the main function with integer local variable cout with initial value.
   e. Display all variables defined above and display the output of cout() function in userDefined namespace and cout() global function using the following description:
      i. … <<"The local variable cout in main is"<<...
      ii. … <<"The variable in userDefined namespace is"<<...
      iii. …<<"The output of cout() in userDefined is"<<...
      iv. …<<"The value of myGlobal is"<<...
      v. …<<" The output of global cout() is"<<...

Omitting Namespace

You might see some C++ programs that runs without the standard namespace library. The `using namespace std` line can be omitted and replaced with the `std` keyword, followed by the `::` operator for some objects:

Example

```
#include <iostream>

int main() {

  std::cout << "Hello World!";

  return 0;

}
```

C++ Constants

In C++, constants may be classified into **literal constants and symbolic constants.**

a) Literal Constants

Literals constants are used to express particular values within a program. For example, in the following statement, 25 is a literal constant because you can neither assign another value to it nor can you change it.

x + **25**;

Literal constants can be classified into integer numerals, floating-point numerals,characters, strings and boolean constants.

b) Symbolic Constants

A symbolic constant is a constant that is represented using a symbolic name. Once a symbolic constant is initialised, its value cannot be changed. There are two ways to declare a symbolic constant in C++ are:

1. Using preprocessor directive #define. Following is the form to use #define preprocessor to define a constant −
   #define identifier value
   For example, the following statement declares a symbolic constant named **numberOfDistricts** that is replaced by 30 during Execution:
   #define numberOfDistricts 30;
2. Using keyword const followed by the data type of the symbolic constant as shown in the statement below:
   For example,
   const short int numberOfDistrict = 30;
   The advantage is that the compiler is able to determine data type of the constant hence preventing possible runtime errors

Exercise

1. Build a c++ program that compute the circumference and area of the circle
   a. Declare a double constant which contains PI=3.14159265
   b. Request the user to enter the radius of a circle and compute the area and circumference of the circle.

Output formatting

In programming, creating nicely formatted output is a good programming practice to improve readability of output and the user interface. In C++, the output stream has special characters and objects called **manipulators** in <iostream> used to format numbers, characters and strings.

a) The endl manipulator

When supplied with operator << at the end of a statements, endl object causes a newline

character to be inserted at the end of a line.

cout << “Hello, world!” << endl;

2 The setw() manipulator

To produce number and string output formatted to fixed width in terms of number of character, C++ has a manipulator object called setw() inside <iomanip> library.. For example, setw(20) in the statement below adjusts the value of **num.** If the characters are fewer, a blank space is inserted on the left of the output.

```
#include <iomanip>
#include <iostream>
using namespace std;
int main()
{
    // Initializing the integer
    int num = 50;
    cout << "Before setting the width: \n"
         << num << endl;
    // Using setw()
    cout << "Setting the width"
         << " using setw to 20: \n"
         << setw(20);
    cout << num << endl;
    return 0;
}
```

```
Before setting the width:
50
Setting the width using setw to 20:
                  50
```

1. setw() is library function in C++.
2. setw() is declared inside #include<iomanip>
3. setw() will set field width.
4. setw() sets the number of characters to be used as the field width for the next insertion operation.

Set 5 characters

cout<<**setw(5)**<<"RCA"<<endl;

2 Fields are blank

3 The setprecision() manipulator

In C++, formatting floating point numbers may be rounded off to the nearest integer using setprecision() manipulator. The object is used together with fixed or scientific manipulators to specify the number of digits to be displayed.

```
#include <iostream>
#include <iomanip>
using namespace std;
int main(){
cout << setw(9) << 8.25 << endl;
cout << setw(20)<< “Hello!”<< endl;
cout<<setprecision(2)<<fixed<<1234.56789<< endl;
cout<<setprecision(3)<<scientific<<1234.56789<< endl;
return 0;
}
```

```
     8.25
                Hello!
1234.57
1.235e+03
```

4. Format Base of Integer Output

In computing, the commonly used numeric constants are decimal integers, floating point (real numbers), octal (base 8) and hexadecimal (base 16). In C++ we use format specifier to format or convert a number from one base to another. To change the base of printed values use dec , oct , and hex manipulators. The following program demonstrates how to format the three number systems:

```
#include <iostream>
using namespace std;
int main(){
int value = 65;
char letter = 'B';
cout << "The following is display of formatted
output" << endl;
cout << "decimal: " << dec<<value << endl;
cout << "octal:" << oct<<value << endl;
cout << "hexadecimal"<< hex<<value <<endl;
return 0;
}
```

```
The following is display of formatted output
decimal:65
octal:101
hexadecimal41
```

5. Format Output using Escape Sequence

Output formatting can also be accomplished using a combination of a backslash and a character, known as escape sequence.

Escape	Meaning	Description
\n	New line	Forces the cursor or insertion pointer to move to a new line
\t	tab	Moves tabs horizontally to create uniform white spaces between outputs.
\b	backspace	Move the character backwards withouterasing anything.
\v	Vertical tab	Moves tabs vertically to create uniform white spaces between outputs.
\r	Carriage return	Moves the cursor to the first column of the net line.
\f	form feed	Moves the cursor to the start on next page.

example

Program

```
# include <iostream>
using namespace std;
int main(){
cout << "Name\t orange\t mango\t apple \n";
cout << "John :\t 3 \t 5 \t 8"<< endl;
cout << "Janet:\t 4 \t 5 \t 7"<< endl;
cout <<"Peter:\t 5 \t 3 \t 6"<< endl;
return 0;}
```

output

```
Name     orange  mango   apple
John :   3       5       8
Janet:   4       5       7
Peter:   5       3       6
```

Solve Complex Problems using C++ functions

In C++, the smallest component having independent functionality is known as a function.

Every C++ program has at least one function called main ( ) through which other functions interact with each other directly or indirectly. This
interaction is made possible through function calls and parameter passing discussed later in this unit.

1.1 Features of C++ Functions

Like in other structured programming languages, the following are characteristics of C++ functions:

- A function is a complete sub-program in itself that may contain input, processing and output logic.
- A function is designed to perform a well defined task.
- A function can be compiled, tested and debugged separately without the intervention of other functions.
- A function has only one entry and one exit point.
- A function can interact with other functions using a mechanism known as function call and parameter passing.
- A function is designed in such a manner that it can be used with different programs or software system.
- The calling function is suspended during the execution of the called function.This implies that there is only one function in execution at any given time.
- Control is always returned to the caller when the function execution terminates.

2 Types of Functions

Functions may be classified into two categories namely: Library or (built-in)functions and user-defined functions. Library functions are compiled and put in C++ library to simplify programming task while user-defined function are the functions that we write to create a modular program.

2.1 Library functions

So far, we have been writing programs by first including (importing) functions from C++ Standard Library. C++ Standard Library provides a collection of predefined functions for common input and output manipulation, calculations, error checking and many other useful operations.

To use a library function, we first include its header file, then use a function that passes list of arguments from the calling portion of the program.

For example, to find the square root of a number, we use square root function sqrt() as follows:
root = sqrt(16);

The function sqrt() evaluates the square root of 16 and returns 4 which is then assigned to the root. In this section, we demonstrate how to use mathematical, string and character manipulation functions.

2.1.1 Math Functions

The C++ Library provides a collection of Math functions used to perform mathematical and trigonometric computations.

For example, to raise 5 to power 3, we use the pow() function as follows:

power = pow(5,3);//returns 125

Next slide enumerates frequently used functions that require inclusion or importing of <cmath> or <math.h> header file using #include directive.

Function	Description	Example
ceil(x)	rounds x to the smallest integer that is greater than or equal to x (rounds up the nearest integer).	ceil(9.2) is 10. ceil (-9.8) is -9. X is a floating value
cos(x)	cosine of x (x in radians)	cos(0.0) is 1
exp(x)	exponential function	exp(1.0) is 2.718282
fabs(x)	absolute value of x	fabs(5.0) is 5.0. fabs(-8.76) is 8.76
floor(x)	rounds x to the largest integer that is smaller than or equal to x (rounds downs the nearest integer).	floor (9.2) is 9. floor(-9.8) is -10 X is a floating value
fmod(x,y)	remainder of x/y as a floating point	fmod(2.6, 1.2) is 0.2
log(x)	natural logarithm of x (base e)	log(2.718282) is 1.0
log10(x)	logarithm of x (base 10)	log10(100.0) is 2.0
pow(x,y)	x raised to power y (x,y)	pow(2.7) is 128
sin(x)	trigonometric sine of x (x in radians)	sin(0.0) is 0
sqrt(x)	square root of x (where x is a non negative value)	sqrt(9.0) is 3.0
tan(x)	tangent of x (x in radians)	tan(0.0) is 0

fmax and fmin

fmax() and fmin() functions are defined in

cmath header file.

fmax() function: The syntax of this function is:

```
double fmax (double x, double y);
float fmax (float x, float y);
long double fmax (long double x,
long double y);
```

The input to this function are two values of type float, double or long double. The function returns maximum of the two input values.

```
#include <cmath>
#include <iomanip>
#include <iostream>
using namespace std;
int main()
{
    double val;

    // Find maximum value when both the inputs
    // are positive.
    val = fmax(10.0, 1.0);
    cout << fixed << setprecision(4)
         << "fmax(10.0, 1.0) = " << val << "\n";
    // Find maximum value when inputs have
    // opposite sign.
    val = fmax(-10.0, 1.0);
    cout << fixed << setprecision(4)
         << "fmax(-10.0, 1.0) = " << val << "\n";

    // Find maximum value when both the inputs
    // are negative.
    val = fmax(-10.0, -1.0);
    cout << fixed << setprecision(4)
         << "fmax(-10.0, -1.0) = " << val << "\n";

    return 0;
}
```

output

```
fmax(10.0, 1.0) = 10.0000
fmax(-10.0, 1.0) = 1.0000
fmax(-10.0, -1.0) = -1.0000
```

fmin()

fmin() function: The syntax of this function is:

```
double fmin (double x, double y);
float fmin (float x, float y);
long double fmin (long double x,
long double y);
```

The input to this function are two values of type float, double or long double. The function returns minimum of the two input values.

```
// CPP program to show working
// of fmin() function.
#include <cmath>
#include <iomanip>
#include <iostream>
using namespace std;
int main()
{
    double val;
    // Find minimum value when both the inputs
    // are positive.
    val = fmin(10.0, 1.0);
    cout << fixed << setprecision(4)
         << "fmin(10.0, 1.0) = " << val << "\n";
    // Find minimum value when inputs have
    // opposite sign.
    val = fmin(-10.0, 1.0);
    cout << fixed << setprecision(4)
         << "fmin(-10.0, 1.0) = " << val << "\n";
    // Find minimum value when both the inputs
    // are negative.
    val = fmin(-10.0, -1.0);
    cout << fixed << setprecision(4)
         << "fmin(-10.0, -1.0) = " << val << "\
n";

    return 0;
}
```

output

```
fmin(10.0f, 1.0) = 1.0000
fmin(-10.0, 1.0) = -10.0000
fmin(-10.0, -1.0) = -10.0000
```

Exercises

1. Using sqrt() and pow() calculate the hypotenuse of the right triangle.

$$\text{hypo} = \sqrt{a^2 + b^2}$$

2. Print all integer pairs(a,b) greater than 1 and less than 100 that can satisfy hypotenuse rule. Hypotenuse should be also integer. Display the number of pairs found. Treat (3,4) and (4,3) as one pair.

$$\text{hypo} = \sqrt{a^2 + b^2}$$

3. Find the roots of quadratic function

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

2.1.2 Character Functions

Although a computer is a numerical machine, most often, data entered into a computer consist of numbers, characters and strings. The underlying fact is that characters are treated as integers. The C++ Library has in-built functions used to manipulate characters. The functions can be accessed by including <cctype> header file.

For example, to convert a character c from uppercase to lower case, we use the following statement:

letter = tolower(c)

Activity: Prompt the user to enter a character and get it converted to lowercase

Character functions

Function	Description	Example
isdigit(c)	Check whether c is a numeric digit	isdigit(‘5’)//returns 1
isalpha(c)	Check whether c is a letter	isalpha(‘5’)//returns 0
isupper(c)	Check whether c is in uppercase	isupper(‘x’)//returns 0
tolower(c)	Converts c to lowercase	tolower(‘R’)//returns r
toupper(c)	Converts c to uppercase	toupper(‘r’)//returns R
isblank(c)	Returns true if character is a space or tab else returns false.	isblank(‘ ‘) or isblank(‘\t’) return 1
isspace(c)	Returns true if character is a space or tab or whitespace control code (Eg.\n,\r) else returns false	isspace(‘\n’) return 1
iscntrl(c)	Returns true if character is tab or any control codeelse returns false.	iscntrl(‘\t’) return 1

Exercises

1. Build a program that checks the number of spaces in a given string.
2. Build a program that check if an entered character is a digit or alpha letter.
3. Build a word count program which display the following through input provided by the user:
    i. Number of characters without spaces
    ii. Number of characters with spaces
    iii. Number of words
4. Build a program that reverse a given string

Note: Treat String as array of characters

String functions

String is a collection of characters. There are two types of strings commonly used in C++ programming language:

- Strings that are objects of string class (The Standard C++ Library string class)
- Character array or C-strings (C-style Strings)

The string-handling library provides many useful functions for manipulating string data, comparing strings, searching strings for characters and substrings. To use the string manipulation functions, you must include the <cstring><string.h> header file. For example, the following statement returns the number of characters in “My House”String

```
#include <iostream>
#include<string.h>
using namespace std;
int main(){
int count=0;
count = strlen("My House");//
cout <<"Number of characters are: "<<count<<endl; //return 8
return 0;}
```

String vs character array(c-string)

A character array is simply an **array of characters** that can be terminated by a null character.

Example:

char str[11]=”C++”;

char str[ ]=”C++”;

char *str=”C++”;

char str[] = {'C','+','+','\0'};

char str[4] = {'C','+','+','\0'};

String is a class that characterises objects that can be depicted as a stream of characters.

Example: String s=”I like c++”;

The sizeof character array needs to be designated statically. More memory can’t be distributed at run time whenever required.

On string memory is distributed progressively and more memory can be distributed at run time.

Character array are quicker while strings are slower. Character array does not offer a lot inbuilt capabilities to control strings. String class has various functions that permit complex

Examples of Commonly used string functions

Function	Description	Example
strcat(x,y)	Concatenates two strings declared as array of characters. The second string is appended to first string. Char str[]=”This is an”; char st[]=”example”;	strcat(str, st) Return **This is an example**
strcmp(x,y)	Compare two strings Return <0,=0 or >0 depending character value.	strcomp(“he”,”se”) return -1 strcomp(“she”,”se”) return 1 strcomp(“se”,”se”) return 0
strlen (s)	Returns the length of the string. strlen (s) does not work on string variable. It is a c function	strlen(‘him’)// returns 3
s.size()	Return the length of string.	String s=”him”; s.size()//return 3
s.length()	Return the length of string.	String s=”him”; s.length()//return 3
strcpy(x,y)	Copies the second string to first string. It overrides the value of first string.	strcpy(y,x); copy x to y
getline()	This function is used to store a stream of characters as entered by the user in the object memory	getline(cin, x)// x as a stream of characters entered
s.push_back('c');	Insert a character c from the end	String s=”hi”; s.push_back(‘m’);// return him
s.pop_back()	Delete a character at the end of a string	String s=”him”; s.pop_back(s);// return hi

Examples

```
#include <iostream>
#include <cstring>
using namespace std;
int main ()
{
  char str[80];
  strcpy (str,"these ");
  strcat (str,"strings ");
  strcat (str,"are ");
  strcat (str,"concatenated.");
  cout<<(str);
  return 0;
}
```

Output

These strings are concatenated.

```
#include <iostream>
#include <cstring>
using namespace std;
int main()
{
    string str = "Rwanda Coding Academy";
    cout << str.size() << endl;
    cout << str.length() << endl;
    cout << strlen(str.c_str()) << endl;
    cout<<strlen(“Rwanda Coding Academy”)<<endl;
return 0;
}
```

Output:

21
21
21
21

Exercises

1. Using while loop calculate the length of a string without using built-in functions.
2. Write a program that check if a given string is a palindrome with and without built in functions.
3. Write a program that converts the entered string to Uppercase
4. Write a program that remove all spaces in a string and print the resulting string.

Word Guess Game homework /10pts, Deadline 1 November 2024: Console

Specification

You are to develop a word guessing game. When the game starts, the user will select a category of words to guess. After choosingthe category, the game select a random word, which it will not tell the user. The user will then be prompted to choose a letter of the alphabet. If that letter occurs in the word, then the game will display the word with only the correctly guessed letters visible.

E.g. The game selects **elephant** as the word

1) The user chooses the letter e
2) The game displays e_e_____
3) The user chooses the letter t
4) The game displays e_e____t

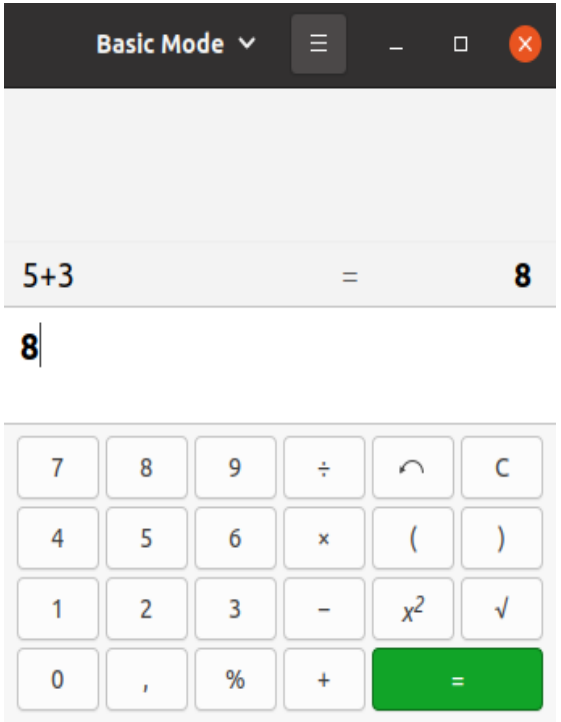
The game will give the user a limited number of chances to guess the word. If the user completely guesses the word before running out of chances, then they win. Otherwise they lose.

- If the user types exit instead of a letter, the program should exit
- Have different categories of words, e.g.**names of animals, teams, districts,films, books**, and allow the user to select a category when the game starts.
- Ask the user if they want to play again when the game is finished

GUI simple Calculator./10 pts

2. Develop a GUI based calculator using **C++** that allows a user to do Addition(+),division(/), subtraction(-) and multiplication(*) of two numbers and show the result in a specified textbox of your calculator.

The sample calculator is displayed on the right. Ignore other functionalities which are not specified.



User Defined Functions

C++ allows programmers to define their own function(s). A user-defined function groups code to perform a specific task and that group of code is given a name(identifier). When the function is invoked from any part of program, it all executes the codes defined in the body of function.

Creating user-defined functions require that you declare the function name, return type, and list of arguments. After the declaration, you can then define the function as follows:

```
type fun_name(arg1,arg2,...){
  statements
    }
```

Example

```
int fun_add(int a, int b){
    int sum;
    sum =a+b;
  return sum;
}
```

Function declaration

C++ requires that a function be defined before being called by the main () function or any other function. For example, addition function in our previous example comes before main. However, if you do not want to fully implement a function, you can first declare it and implement it later.

To declare a function without implementing the body, write the function return type, name and parameter list followed by a semicolon at the end of the statement.

The portion of a function that includes only the function name and list of arguments is called a **function signature or prototype.**

Example:

double maximum(double x,double y,double z);

Functions with no type. The use of void

Previously, we defined the the type for the value returned by the function. But what if the function does not need to return a value? In this case, the type to be used is **void,** which is a special type to represent the absence of value. For example, a function that simply prints a message may**void** can also be used in the function's parameter list to explicitly specify that the function takes no actual parameters when called. For example, printmessage could have been declared as:

```
#include <iostream>
using namespace std;
void printMessage ()
{
  cout << "I'm a function!";
}
int main ()
{
  printMessage ();
}
```

*void printMessage(**void**) // Here void is optional in C++*

In C++, an empty parameter list can be used instead of void with same meaning, but the use of void in the argument list was popularized by the C language, where this is a requirement.

It is mandatory in C++ to show parentheses which follow the function name, its declaration and when calling it. The parentheses are what differentiate functions from other kinds of declarations or statements.

```
printMessage (); // This is a function call
Printmessage;  // This is not a function call
```

Call a function

When a function is called by another function, execution is transferred to the function until the return statement or end of function is encountered. To demonstrate how functions work, the following program calculates the sum of two numbers received from the main function:

```
*this program consists of two
function:
main and addition */
#include <iostream>
using namespace std;
//addition function calculates sum
int addition (int a, int b){
int sum;
sum=a+b;
return sum;
}
//program execution starts here!
int main (){
int total=0,x=5, y=7;
total = addition (x,y);
cout<< “The sum is:”<<total<<endl;
return 0;
```

Output

The sum is 12

Call a function multiple times

A function can actually be called multiple times within a program.

This example defines a subtract function, that simply returns the difference between its two parameters. This time, main calls this function several times, demonstrating more possible ways in which a function can be called.

```
/ Call a function multiiple times
#include <iostream>
using namespace std;
int subtraction (int a, int b)
{
  int r;
  r=a-b;
  return r;
}
int main ()
{
  int x=5, y=3, z;
  z = subtraction (7,2);
  cout << "The first result is " << z << '\n';
  cout << "The second result is " << subtraction (7,2) << '\n';
  cout << "The third result is " << subtraction (x,y) << '\n';
  z= 4 + subtraction (x,y);
  cout << "The fourth result is " << z << '\n';
```

```
The first result is 5
The second result is 5
The third result is 2
The fourth result is 6
```

Exercises

1. Study the code snippet shown below and identify the function's list of parameters, return type and the value returned by the maximum function:

```
double maximum( double x, double y, double z ){
double maxiValue = x; //assume x is maximum
if ( y > maxiValue)
maxiValue = y; // make y the new maximum
if ( z > maxiValue)
maxiValue = z; // make z the newmaximum
return maxiValue;
} // end function
```

Question: Write a complete program in which the maximum function is called from the main ();

2. Write a program that computes area of a rectangle in a function called rect_area. The rectangle then returns the calculated area to the main function.

Overloaded functions

In C++, two different functions can have the same name if their parameters are different; either because they have a different number of parameters, or because any of their parameters are of a different type.

Note that a function cannot be overloaded only by its return type. At least one of its parameters must have a different type.

```
// overloading functions
#include <iostream>
using namespace std;

int operate (int a, int b)
{
  return (a*b);
}
double operate (double a, double b)
{
  return (a/b);
}
int main ()
{
  int x=5,y=2;
  double n=5.0,m=2.0;
  cout << operate (x,y) << '\n'; //call operate with integer arguments
  cout << operate (n,m) << '\n'; //call operate with double arguments
  return 0;
}
```

Overloaded functions(next)

Overloaded functions may have the same definition.Here, sum is overloaded with different parameter types, but with the exact same body.

```
// overloaded functions
#include <iostream>
using namespace std;
int sum (int a, int b)
{
  return a+b;
}
double sum (double a, double b)
{
  return a+b;
}
int main ()
{
  cout << sum (10,20) << '\n';
  cout << sum (1.0,1.5) << '\n';
  return 0;
}
```

Functions template

The function sum() in the previous example could be overloaded for a lot of types, and it could make sense for all of them to have the same body. For cases such as this, C++ has the ability to define functions with generic types, known as *function templates*. Defining a function template follows the same syntax as a regular function, except that it is preceded by the template keyword and a series of template parameters enclosed in angle-brackets <>:

template <template-parameters> function-declaration

The template parameters are a series of parameters separated by commas. These parameters can be generic template types by specifying either the **class or typename** keyword followed by an identifier. This identifier can then be used in the function declaration as if it was a regular type. For example, a generic sum function could be defined as:

```
template <class SomeType>
SomeType sum (SomeType a, SomeType b)
{
 return a+b;
}
```

```
template <typename T>
T sum (T a, T b)
{
 return a+b;
}
```

In the code above, declaring SomeType of T (a generic type within the template parameters enclosed in angle-brackets) allows T to be used anywhere in the function definition, just as any other type; it can be used as the type for parameters, as return type, or to declare new variables of this type. In all cases, it represents a generic type that will be determined on the moment the template is instantiated.

Instantiating a template is applying the template to create a function using particular types or values for its template parameters. This is done by calling the *function template*, with the same syntax as calling a regular function, but specifying the template arguments enclosed in angle brackets:

name <template-arguments> (function-arguments)

For example, the sum function template defined above can be called with:

- ***x=sum<int>(10,20)***

Example

```
#include <iostream>
using namespace std;

template <typename T>
T sum (T a, T b)
{
  return a+b;
}

int main ()
{
  int x=5,y=2;
  double n=5.0,m=2.5;
  cout << sum<int>(x,y) << '\n';
  cout << sum<double>(n,m) << '\n';
  return 0;
}
```

```
7
7.5
```

Function templates with multiple template type parameters

We've only defined the single template type **(T)** for our function template, and then specified that both parameters must be of this same type. Now, if we wants to pass different types of parameters, we can rewrite our function template in such a way that our parameters can resolve to different types. Rather than using one template type parameter **T**, we'll now use two **(T and U):**

```
#include <iostream>
// We're using two template type parameters named T and U
template <typename T, typename U>
// x can resolve to type T, and y can resolve to type U
T max(T x, U y){
// uh oh, we have a narrowing conversion problem here
    return (x < y) ? y : x; }
int main()
{
    std::cout << max(2, 3.5) << '\n';
    return 0;
}
```

To avoid loss of data if 3.5 is to be returned as int while it is double; we can use an `auto` return type -- we'll let the compiler deduce what the return type should be from the return statement:

```
#include <iostream>
template <typename T, typename U>
auto max(T x, U y) {
 return (x < y) ? y : x;
}
int main() {
std::cout << max(2, 3.5) << '\n';
return 0;
}
```

Regular data type and template

Templates are a powerful and versatile feature. They can have multiple template parameters, and the function can still use regular non-templated types. Here below **are_equal<int,double>(10,10.0) is**

```
#include <iostream>
using namespace std;
template <class T, class U>
bool are_equal (T a, U b)
{
  return (a==b);
}
int main ()
{
  if (are_equal(10,10.0))
    cout << "x and y are equal\n";
  else
    cout << "x and y are not equal\n";
  return 0;
}
```

```
x and y are equal
```

The template parameters can not only include types introduced by class or typename, but can also include expressions of a particular type:The second argument of the fixed_multiply function template is of type int.

```
// template arguments
#include <iostream>
using namespace std;
template <class T, int N>
T fixed_multiply (T val)
{
  return val * N;
}
int main() {
  std::cout << fixed_multiply<int,2>(10) << '\n';
  std::cout << fixed_multiply<int,3>(10) << '\n';
return 0;
}
```

```
20
30
```

Recursive function

Recursion in computer science means a function is calling itself.
We will use recursion whenever the solution of a problems depends on the solution of the small problem of the same nature.
Example: Factorial of n
n!=n*n-1*n-2*n-3*n-4*n-5……..*1
n!=n*(n-1)!
fact(n)=n*fact(n-1);
Big=small->smaller->smallest

Recursion and Principle of Mathematics Induction(PMI)

The Principle of Mathematical Induction (PMI) is a theorem that gives a method for establishing the truth of statements quantified over all integers greater than or equal to some given integer.

The Principle of Mathematical Induction (PMI) is just the following observation. Let P(n) be a statement for each positive integer n. If P(1) is true and if P(k) is true the prove that P(k +1) is true for all positive integers k, then P(n) is true for all positive integers n. In other words, if P(1) and P(k) ⇒ P(k + 1) then P(1) ⇒ P(2) ⇒ P(3) ⇒ P(4) ⇒ · · · .

It uses three main steps

1. **A Basis or Base case**
2. **Induction Hypothesis which is an assumption**
3. **Induction Step which use the induction hypothesis to argue for all value of n**

In computer science, PMI helps in the analysis of algorithms. But not only that,proofs by induction also tend to imply recursive algorithms for solving the problem at hand. **PMI is a main tool in proving the correctness of recursive algorithms.** The recursion works on the Principle of Mathematics Induction which is used to prove facts.

Recursion Principle of Mathematics Induction

The recursion works on the Principle of Mathematics Induction which is used to prove facts. It has only 2 steps:

- ***Step 1. Show it is true for the first one***
- ***Step 2. Show that if any one is true then the next one is true***

Then all are true

In the world of numbers we say:

- Step 1. Show it is true for first case, usually **n=1**
- Step 2. Show that if **n=k** is true then **n=k+1** is also true

PMI Example:Sum of Numbers from 1 to n

$$\sum_{i=1}^{k} i = \frac{k(k+1)}{2}$$

Prove that ∑n=n(n+1)/2

1. **Best case**

Let us think n=0,

Left hand side is ∑0=0

Right Hand Side is n(n+1)/2=0(0+1)/2=0

When n=1, LHS is ∑1=1 , the RHS is 1(1+1)/2=2/2=1

2. **Induction Hypothesis(Assumption)** that ∑k=k(k+1) /2 is true
3. **Induction step**

   ∑(k+1)=(k+1)(k+2)/2

   LHS =∑(k+1)

   (k+1)+∑k then replace with the value of ∑k

   (k+1)+k(k+1)/2=(k+1)*2/2+k(k+1)/2

   =(k+1)(k+2)/2

   LHS=RHS Hence proved

Let $n = k + 1$. Now

$$\begin{aligned}
\sum_{i=1}^{n} i &= \sum_{i=1}^{k+1} i \\
&= \left(\sum_{i=1}^{k} i\right) + (k+1) \\
&= \frac{k(k+1)}{2} + (k+1) \\
&= \frac{k(k+1) + 2(k+1)}{2} \\
&= \frac{(k+1)(k+2)}{2} \\
&= \frac{n(n+1)}{2}.
\end{aligned}$$

PMI Example 2: Prove that 3^n-1 is a multiple of 2?

Example: is 3^n-1 a multiple of 2?

Is that true? Let us find out.

1. Show it is true for **n=1**

$$3^1-1 = 3-1 = 2$$

Yes 2 is a multiple of 2. That was easy.

3^1-1 is true

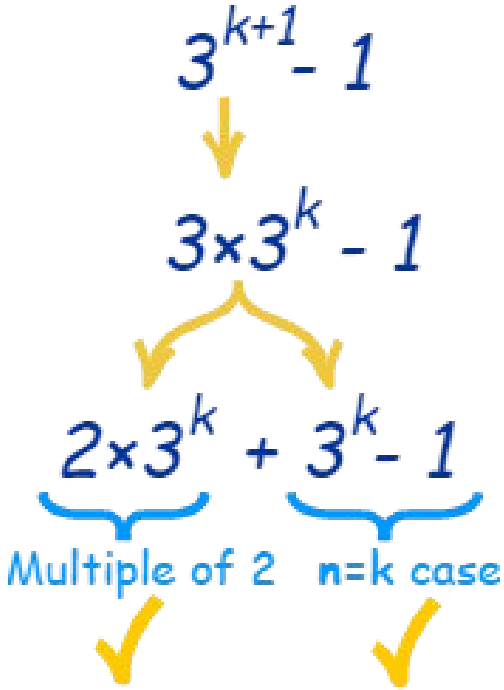
2. Assume it is true for **n=k**

3^k-1 is true

(Hang on! How do we know that? We don't!

It is an **assumption** ... that we treat

as a fact for the rest of this example



Recursion example: Factorial of a number

```
#include<iostream>
using namespace std;

int fact(int n){
    ///cout<<n<<endl;
    // Step1: Best Case
    if(n < 0) return -1;
    if(n==0) return 1;
// Step 2: Assumption
     int smallerAnswer = fact(n-1);
       // Step 3: Calculation
     return n*smallerAnswer ;
}

int main(){
   int n=5;

     int answer = fact(n);
     cout<<answer <<endl;
    return 0;
}
```

Note: The two lines in bold are added to avoid the segmentation fault which occurs when memory is exhausted(Lack of memory) due to infinite calls:

```
if(n < 0) return -1;
if(n==0) return 1;
```

Fibonacci number at n position

Each number is equal to the sum of the preceding two numbers. The Fibonacci sequence begins with the following 14 integers: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, **55,** 89, 144, 233.

Ex: f(10)=f(9)+f(8)=55

f(n)=f(n-1)+f(n-2) which is a recursion as the big problem is solved with solution of the small problems.

```
#include<bits/stdc++.h>
using namespace std;

int fib(int n){
   /// Base case
   if(n==0){
    return 0;
   }
   if(n==1){
    return 1;
   }
   // Recursive case
   int smallOutput1 = fib(n-1);
   int smallOutput2 = fib(n-2);
   /// calculation
   return smallOutput1 + smallOutput2;
}

int main(){
    cout<<fib(3);
    return 0;
}
```

Power of a number

To calculate the x to the power of n;
we can use the recursive case,

$x^n = x * x^{n-1}$

Base case

If n=0, $x^0 = 1$

Calls	(5,0)	(5,1)	(5,2)	(5,3)
Output	1	5	25	125

```
#include<bits/stdc++.h>
using namespace std;

int power(int x,int n){
  ///Base Case
  if(n==0){
    return 1;
  }
  ///Recursive
  int smallOutput = power(x,n-1);

  /// Calculation
  return x*smallOutput;
}

int main(){
     cout<<power(5,3);
    return 0;
}
```

Sum of n numbers

How to calculate the sum of n numbers using recursion.
Let say 12345
sum(n)=sum(n/10)+last number

```
#include<bits/stdc++.h>
using namespace std;

int sum(int n){
    /// base case
    if(n==0){
        return 0;
    }
    /// recursive
    int smallAns = sum(n/10);

    /// calculation
    int last_digit = n%10;
    return smallAns + last_digit;
}
int main(){
    cout<<sum(12345);
    return 0;
}
```

Print numbers from 1 to n

Using recursive algorithms, write a function to print numbers from 1 to n.

If n is 3 the function will print

1 2 3

If n is 5 the function will print

1 2 3 4 5

If n is 10 the function will print

1 2 3 4 5 6 7 8 9 10

print(n)=print(n-1)+n;

If the value of n is zero we will not print

If the value of n is 1 we will print 1

```
#include<bits/stdc++.h>
using namespace std;
void print(int n){
   /// Base case
   if(n==0){
      return;    ///  mandatory printing nothing to avoid infinite loop
   }
   /// Recursive case
   print(n-1);     ///  1 2 3 4 ......n-1
   cout<<n<<endl;
    return;    /// optional
}
//printing in reverse order like 54321
void print2(int n){
   /// Base case
   if(n==0){
      return;    /// mandatory printing nothing to avoid infinite loop
   }
   cout<<n<<endl;
   /// Recursive case
   print2(n-1);     ///n-1.....2 1
    /// return;    optional
}
int main(){
     print2(5);
    return 0;
}
```

Count Digits of a number

Build a function which counts number of a given digit which greater than or equal to 1.
If the number is 45756, the function returns 5
If the number is 900, the function returns 3
If the number is 15, the function returns 2
4125 will be broken into 412 and 5
Then 412 will be broken into 41 and 2
Then 41 will be broken into 4 and 1
Then 4 will be broken into 0 and 4, where 4 is our base case.
count(n)=count(n/10)+1, This would be our recursive case

```
#include<bits/stdc++.h>
using namespace std;

int count(int n){
   /// base
   if(n==0){
    return 0;
   }
   /// recursive
   int smallAns = count(n/10);

   /// calculation
   return smallAns + 1;

}

int main(){
     cout<<count(7820);
    return 0;
}
```

Multiplication of m by n

Build a function to count the power of m by n. M and n must be greater than zero. If m is zero it will be treated as our base case

```
#include<bits/stdc++.h>
using namespace std;

int multiply(int m,int n){    /// m*n
    /// base
    if(n==0){
        return 0;
    }
    /// recursive
    int smallAns = multiply(m,n-
1);  /// m*(n-1)

    /// calculation
    return smallAns + m;
}

int main(){
    cout<<multiply(3,5);
    return 0;
}
```

Counting the number of Zeros in a given number

Build a function to calculate the number of zeros in a number n provided.

For example n=12050, countZeros(n)=2

Which means

countZeros(n)=countZeros(n/10)

if(lastNumber =0)

Return smallAns+1;

Else

Return smallAns;

```
#include<bits/stdc++.h>
using namespace std;

int countZeroes(int n){
  /// base
  if(n==0){
    return 0;
  }
  /// recursive
  int smallAns = countZeroes(n/10);

  /// calculation
  int last_digit = n%10;

  if(last_digit==0){
    return 1 + smallAns;
  }else{
    return smallAns;
  }

}

int main(){
    cout<<countZeroes(10320);
    return 0;
}
```

Check if the array is sorted

Base case: if the number of elements is zero or 1, the array is sorted. Check if the first element is greater than the second , if so return false.
If true; use the recursion for **small array** starting on the second index. The table on the right indicates the small arrays on each recursion by moving the **pointer** in our main array.
If any small array is not sorted, the big array is not sorted.

0	1	2		
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5

C++ Program to check if the array is sorted.

```
#include<iostream>
using namespace std;
bool isSorted(int a[], int n){
   if(n==0 || n==1){
    return true;
   }
   if(a[0] > a[1]){
    return false;
   }
   return isSorted(a+1 , n-1);;
}
// Alternatively
bool isSorted2(int a[],int n){
   if(n==0 || n==1){
       return true;
   }
   if(a[n-2] > a[n-1]){
    return false;
   }
   return isSorted2(a,n-1);
}
```

```
int main(){
  int a[] = {1,2,3,4,5};
  if(isSorted(a,5)){
    cout<<"Sorted"<<endl;
  }else{
    cout<<"Not Sorted"<<endl;
  }
  return 0;
}
```

Sum of the array using recursion

The return type is integer as the sum of the array.
We are going to break the problem into smaller problem

a[0]	small	ar	ra	y

We will return the sum of small array in green and add the first element

```
#include<iostream>
using namespace std;

int sumOfArray(int a[], int n){
Base case
   if(n==0){
    return 0;
   }

   return a[0] + sumOfArray(a+1 , n-1);
}

int main(){
  int a[] = {1,2,3,4,5};
  cout<<sumOfArray(a, 5)<<endl;;
  return 0;
}
```

Count the Occurence of the number in the array

The next question is to find the occurrence of an element **x** in the array of **n** elements from a given index **i**, Suppose array of {3,3,4,3,5,3}, the occurrence of 3 is 4 since it is in array four times.

The question can be solved using for loop but what is we want to solve the same Question using the recursion?

```
#include<iostream>
using namespace std;
// Answer will be passed as a reference variable
initially zero //The function has return type of void
void count(int a[],int n,int x,int i, int &ans){
//Base case
    if(i==n){
      return;
   }
   if(a[i]==x){
    ans++;
   }
   count(a,n,x,i+1,ans);
}
int main(){
  int a[] = {5,5,6,5,6,7};
  int ans = 0;
  count(a,6,10,0,ans);
  cout<<ans<<endl;
  return 0;
}
```

Check if the number is palindrome

A palindrome is a number if read from left to right or right to left it will be the same.

Example : a=12321

A is a palindrome because from left to right the number is still the same.

In the recursion if a[start]=a[end] the array is palindrome

Then perform the recursion for small array

a[0]	small	arr	ay	a[n-1]

```
#include<iostream>
using namespace std;
//s as start index, e as end index
bool checkPal(int a[],int s,int e){
//Base case for empty array
    if(s > e){
        return true;
    }
    if(a[s] == a[e]){
        return checkPal(a,s+1,e-1);
    }else{
      return false;
    }
}

int main(){
  int a[] = {1,2,3,4,3,2,1};
  cout<<checkPal(a,0,6);
  return 0;
}
```

Exercises: Using recursion, solve the following Questions

1. Write a program with a function that print the character array.

   Hint: if **abc** is provided, the program prints **abc.** Print the first character and call recursion on small array. If the character is '\0' return.

2. Write a program with a function that print the reversed character array.

   Hint: if **abc** is provided, the program prints **cba.** Print the last character and call recursion on small array. If the character is '\0' return.

3. Remove a character in the array.

   Hint: if **abacada** is provided, by removing **a** the program print **bcd**

4. Replace the character in the array

   Hint: if **abacada** is provided, by replacing **a** with **x** the program print **xbxcxdx**

5. Find the length of Character Array

   Hint: if **abcde** i Finds provided, the length should be **5**

6. Convert the String of digits to Integer.

Hint: if "**12345**" is provided then the program would return **12345**

Hint:

If we have "12345" ,
Then call the recursion on "1234",
Then call the recursion on "123",
Then call the recursion on "12",
Then call the recursion on "1", now n=1,
Then call the recursion on empty array.

1. if n=0, then return 0;
2. The small answer is 0, then 0*10+1 as the last digit which is 1
3. Then 1*10+2 as the last digit
4. Then 12*10+3
5. Then 123*10+4
6. Finally 1234*10+5

Exercises(next: Use recursion)

7. A function that remove consecutive duplicate in the string

Examples:

if aabbbccdd the function would print abcd

if abcde is provided, the function would give abcde

8. Find the last index of an element in the array

a[6]={5,5,6,20,5,6}

if x=5, the last index of x is 3.

int lastIndex(int arr[], int element)

8. Find the first index of an element in the array

a[6]={5,5,6,20,5,6}

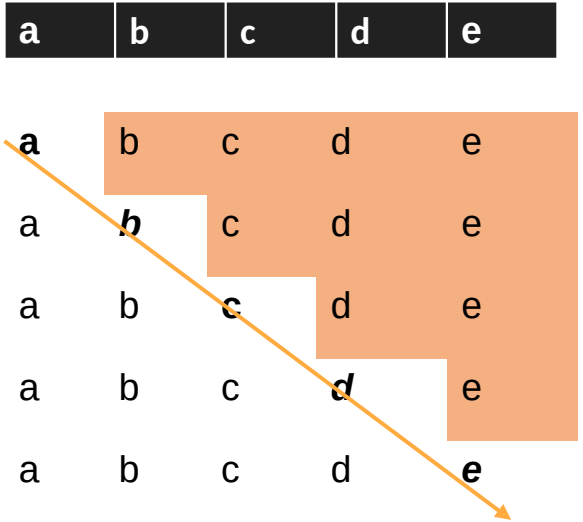
if x=5, the first index of x is 0.

int firstIdex(int arr[], int element)

Recursion on Character Array: Print array and Print reversed array

- Let us think on two functions that (1) print a string/character array and (2) a reversed string . If we have original string **abc** the reverse will be **cba.**
- **Print Function**
- Let us start with first function
  - The best case for first function is when the array is empty. The first character will be equal to zero i.e arr[0]='\0'
  - If the array is empty, we will not print anything.
  - If the array is not empty, we will print the first element.
  - Then call print recursion

Design in table

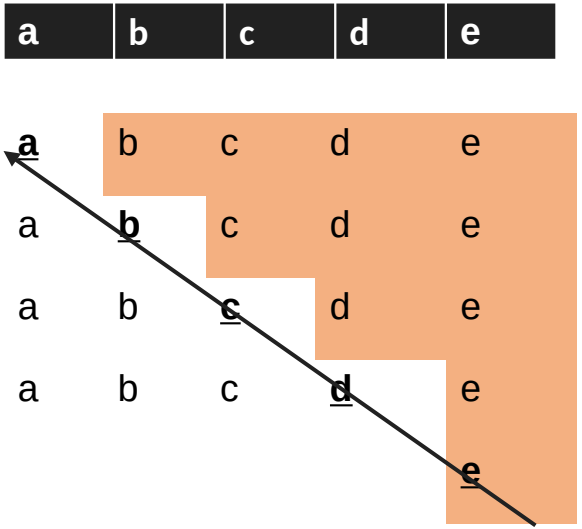



```
#include<iostream>
using namespace std;
void print(char input[]){
//If the array is zero,  we don't print
    if(input[0]=='\0'){
        return ;
    }
//Print the output
    cout<<input[0];
// Call recursion
    print(input+1);
}
int main(){
  char input[] = "RCA";
  print(input);
return 0;
}
```

Function 2: Recursion on Reversing Character Array

- Let us start with second reverse function
- We first print the small array and the print the first value. The order will be reversed.

Design in table

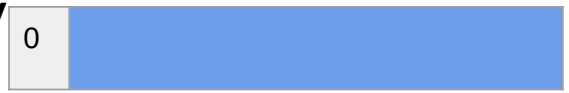



```
#include<iostream>
using namespace std;

void revPrint(char input[]){
     if(input[0]=='\0'){
       return ;
    }
   revPrint(input+1);
   cout<<input[0];
}

int main(){
  char input[] = "abc";
revPrint(input);
  return 0;
}
```

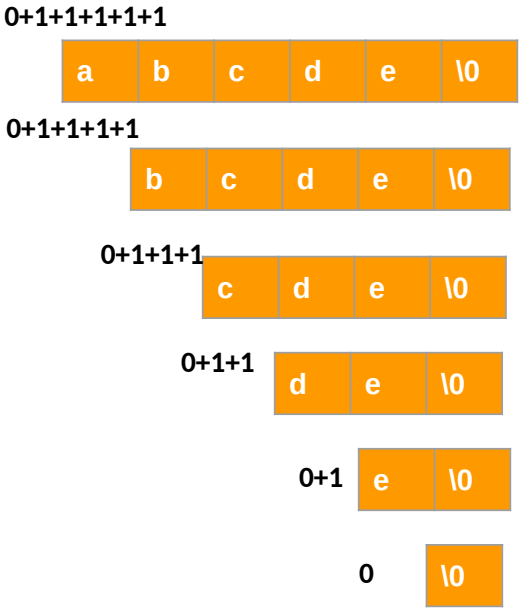
Recursion on Finding length of Character Array

We can find the length of the character array using loops but we need to apply recursion.
If the character array is **abcde** the length should be **5**

Array



1+length
For the recursion, we need to call the length of the small array plus 1 of the first element at index 0.
The base case is when the array is empty, the function should return 0
If not, we calculate the value of length of the small array plus 1




```
#include<iostream>
using namespace std;

int length(char input[]){
    if(input[0]=='\0'){
        return 0;
    }
    return 1 +
length(input+1);
}

int main(){
  char input[100];
  cin>>input;
  int l = length(input);
  cout<<l<<endl;
  return 0;
}
```

Replace a character recursively

Basically, to replace the character **a** by **c** in ababa, means we need to modify the string to have **c**b**c**b**c**

Simply, I need to :

- Check if the first character is **a**
- Replace the character a with **x**

  input[0]=’a’

  input[0]=’c’;

  Then call the recursion for the small array by moving the pointer

```
void replaceCharcter(char input[], char s, char e){
    if(input[0]=='\0'){
        return;
    }
    if(input[0]==s){
        input[0] = e;
    }
    replaceCharcter(input + 1);

}
```

Remove a character in array Recursively

The next problem is to remove a character in the array. The array is **abacada**, then we want to remove the character **a** in the array to have **bcd**.

abacada\0=> bcd\0

if the first character is the target, we will shift the elements, otherwise we will do the recursion

```
void removeC(char input[], char c){
   if(input[0]=='\0'){
    return ;
   }
   if(input[0]!= c){
    removeC(input + 1);
   }else{
      for(int i=0;input[i]!='\0';i++){
        input[i] = input[i+1];
      }
      removeC(input);
   }
}
```

Convert a string digits to Integer

Convert a string “12345” to 12345. This can be achieved using loops but the current target is the use of recursion with the following:

1. if n is 0, then return 0.
2. Calculate the small answer by calling a function on small string by moving the last index.

   convertStrToInt(str,n-1);
3. Calculate the last digit

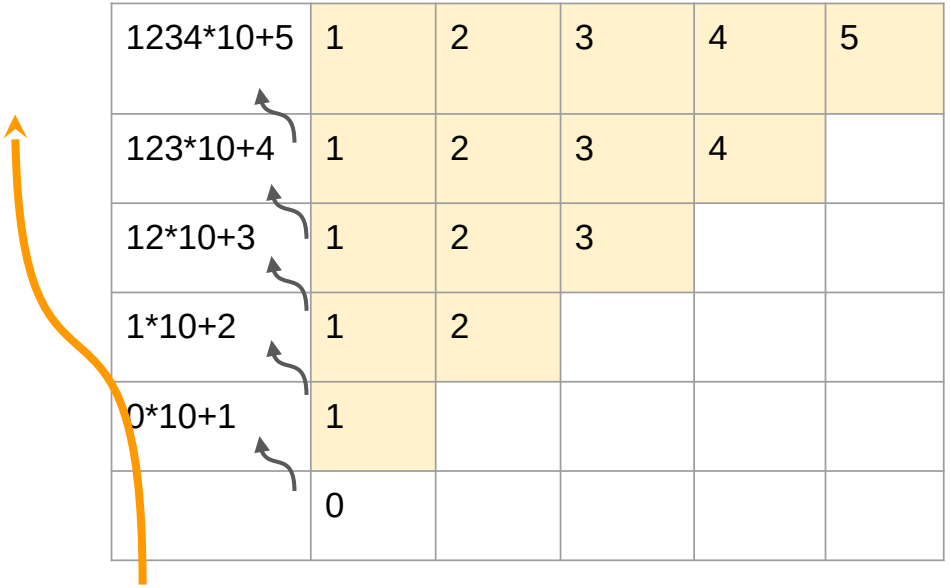
   lastDigit=str[n-1]-’\0’
4. Multiply the smallAnswer by 10 and add last dit to find the answer.
5. smallAnswer*10+lastDigit

```
int convertStringToInt(char str[], int n){
    if(n==0){
       return 0;
    }
    int smallAns = convertStringToInt(str, n-1);
//Character minus a character equal integer
    int lastDigit = str[n-1] - '0';
    int ans = smallAns*10 + lastDigit;
    return ans;
}
```

String to Integer: Let us analyse our code

If we have “12345” , Then call the recursion on “1234”, then call the recursion on “123”, then call the recursion on “12”, then call the recursion on “1”, now n=1, then call the recursion on empty array.

1. if n=0, then return 0;
2. The small answer is 0, then 0*10+1 as the last digit which is 1
3. Then 1*10+2 as the last digit
4. Then 12*10+3
5. Then 123*10+4
6. Finally 1234*10+5



Note that the code will work for specific input like the example provided but not string like “123abc56” as well as string with no more 9 digits.

Exercises on Functions

Using Functions solve the problems in the below file.

https://shorturl.at/ACDPW

Introduction to Object Oriented Programming in C++

Introduction to OOP

OOP(Object Oriented Programming) is a programming style that involves class and object. An object is a basic entity that possess some properties and functions. In real world, we are surrounded with Objects.

For example laptop is an object, chair is an object, table is an object.

We wants to involve object in our code because we want our code to be close to the real world.

Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

A class

A class is a user-defined type which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A class is defined also as blueprint or a **template** for **creating object.**

It is composed of built in types, other user defined types and functions. The parts used to define class are called members. A class has zero or more members. For example

```
class X{
public:
Int m; // data member
Int mf(int v){ // function member
Int  d=m;
m=v;
return d;
};
```

Members are **data members and member Functions**. Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions define the properties and behaviour of the objects in a Class.. We access members using **object.member** notation. For example:

```
X var; //var is a variable of type x
var.m=7; //assign to var's data member to m
Int x=var.mf(9);// call var's member function mf()
```

You can read **var.m** as **var's m.** Most people pronounce var dot m or var's m. A member function **X's mf()** does not need to use the var.m notation, It can use the plain member m as per previous definition int mf(int v){int d=**m**;..}

Defining a class

Classes are defined using either keyword `class` or keyword `struct`, with the following syntax:

```
class class_name {
  access_specifier_1:
    member1;
  access_specifier_2:
    member2;
  ...
} object_names;
```

Where `class_name` is a valid identifier for the class, **`object_names`** is an optional list of names for objects of this class. The body of the declaration can contain ***members***, which can either be **data** or **function** declarations, and optionally *access specifiers*.

Classes have the same format as **plain *data structures***, except that they can also include functions and have ***access specifiers*.** An ***access specifier*** is one of the following three keywords: `private`, `public` or `protected`. These specifiers modify the access rights for the members that follow them:

- `private` members of a class are accessible only from within other members of the same class (or from their *"friends"*).
- `protected` members are accessible from other members of the same class (or from their *"friends"*), but also from members of their derived classes.
- Finally, `public` members are accessible from anywhere where the object is visible.

Class definition(next)

By default, all members of a class declared with the `class` keyword have private access for all its members. Therefore, any member that is declared before any other *access specifier* has private access automatically. For example:

```
class Rectangle {
    int width, height;
  public:
    void set_values (int,int);
    int area (void);
} rect;
```

Declares a class (i.e., a type) called **`Rectangle`** and an object (i.e., a variable) of this class, called **`rect`**. This class contains **four members**: two data members of type `int` (member `width` and member `height`) with *private access* (because private is the default access level) and two member functions with *public access*: the functions `set_values` and `area`, of which for now we have only included their declaration, but not their definition.

Notice the difference between the *class name* and the *object name*: In the previous example, `Rectangle` was the *class name* (i.e., the type), whereas `rect` was an object of type `Rectangle`. It is the same relationship `int` and a have in the following declaration:

```
int a;
```

After the declarations of `Rectangle` and **`rect`**, any of the public members of object **`rect`** can be accessed as if they were normal functions or normal variables, by simply inserting a dot (`.`) between ***object name*** and ***member name*****.** This follows the same syntax as accessing the members of plain data structures. For example:

```
rect.set_values (3,4);
myarea = rect.area();
```

The only members of `rect` that cannot be accessed from outside the class are **`width`** and **`height`**, since they have private access and they can only be referred to from within other members of that same class.

Object

An Object is an identifiable entity with some characteristics and behaviour. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated. Classes are an expanded concept of data structures: like data structures, they can contain data members, but they can also contain functions as members. An object is an instantiation of a class. In terms of variables, a class would be the type, and an object would be the variable.

For example:

```
class Rectangle {
    int width, height;
  public:
    void set_values (int,int);
    int area (void);
} rect; // rect is an object
```

```
class Person
{
        char name[20];
        int id;
public:
        void getdetails(){}
};
int main()
{
Person p1; // p1 is a object
}
```

Implementation of Rectangle class

```
#include <iostream>
using namespace std;

class Rectangle {
    int width, height;
  public:
    void set_values (int,int);
    int area() {return width*height;}
};

void Rectangle::set_values (int x, int y) {
  width = x;
  height = y;
}

int main () {
  Rectangle rect;
  rect.set_values (3,4);
  cout << "area: " << rect.area();
  return 0;
}
```

```
area: 12
```

Explained:

This example reintroduces the *scope operator* (`::`, two colons), seen in earlier chapters in relation to namespaces. Here it is used in the definition of function **`set_values`** to define a member of a class outside the class itself.

Notice that the definition of the member function **`area`** has been included directly within the definition of class **`Rectangle`** given its extreme simplicity. Conversely, **`set_values`** it is merely declared with its prototype within the class, but its definition is outside it. In this outside definition, the operator of scope (`::`) is used to specify that the function being defined is a member of the class `Rectangle` and not a regular non-member function.

The scope operator (`::`) specifies the class to which the member being defined belongs, granting exactly the same scope properties as if this function definition was directly included within the class definition. For example, the function **`set_values`** in the previous example has access to the variables `width` and `height`, which are private members of class `Rectangle`, and thus only accessible from other members of the class.

The only difference between ***defining a member function completely within the class definition or to just include its declaration in the function and define it later outside the class***, is that in the first case the function is automatically considered an *inline* member function by the compiler, while in the second it is a normal (not-inline) class member function. This causes no differences in behavior, but only on possible compiler optimizations.

Members **`width`** and **`height`** have private access (remember that if nothing else is specified, all members of a class defined with keyword `class` have **private** access). By declaring them private, access from outside the class is not allowed.

This makes sense, since we have already defined a member function to set values for those members within the object: the member function **`set_values`**. Therefore, the rest of the program does not need to have direct access to them.

Multiple objects

The most important property of a class is that it is a type, and as such, we can declare multiple objects of it. For example, following with the previous example of class `Rectangle`, we could have declared the object `rectb` in addition to object `rect`:

```
// example: one class, two objects
#include <iostream>
using namespace std;

class Rectangle {
    int width, height;
  public:
    void set_values (int,int);
    int area () {return width*height;}
};

void Rectangle::set_values (int x, int y) {
  width = x;
  height = y;
}

int main () {
  Rectangle rect, rectb;
  rect.set_values (3,4);
  rectb.set_values (5,6);
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;
}
```

```
rect area: 12
rectb area: 30
```

The class (type of the objects) is `Rectangle`, of which there are two instances (i.e., objects): `rect` and `rectb`. Each one of them has its own member variables and member functions.

Notice that the call to **`rect.area()`** does not give the same result as the call to **`rectb.area()`.** This is because **each object of class `Rectangle` has its own variables `width` and `height`**, as they -in some way- have also their own function members `set_value` and `area` that operate on the object's own member variables.

Classes allow programming using object-oriented paradigms: **Data and functions are both members** of the object, reducing the need to pass and carry handlers or other state variables as arguments to functions, because they are part of the object whose member is called. Notice that **no arguments were passed on the calls to `rect.area` or `rectb.area`**. Those member functions directly used the data members of their respective objects `rect` and `rectb`.

Encapsulation

In C++, encapsulation involves combining similar data and functions into a single unit called a class. By encapsulating these functions and data, we protect that data from change. This concept is also known as data or information hiding. Let's look at an example of C++ class encapsulation:

```
...
class Passport {
string country_of_origin;
int passport_number;
string passport_name;
void dothis() {
    cout << passport_name << "'s " << country_of_origin << " passport number is " <<
passport_number;
}
};...
```

Within the class `Passport`, we declared the variables and methods that we wanted to include. Note that we can only use `cout()` if we wrap it in a function, as classes can execute member functions but not objects.

Advantages of Encapsulation

Keeping pertinent code encapsulated within a class helps streamline your program. You can contain your code in one location so your code is easier to follow.

This, in turn, makes your program easier to debug. If a function within your encapsulated class is not working correctly, you'll likely be able to pinpoint the problem to the class instead of having to comb through your code.

When you encapsulate, you can control the flow of data into and out of a class. Classes allow you to make certain functions or variables inaccessible to protect them from being used or changed outside of the class. You can also make specific components accessible to use elsewhere in your code.

Let's look at the tools that allow you to protect or enable access to a class's variables or functions.

```
..class Passport {
private:
  string country_of_origin;
public:
  int passport_number;
  string passport_name;
  void dothis() {
        cout << passport_name << "'s " << country_of_origin << " passport number is " << passport_number;
  }
};...
```

Building on our example, we declare `country_of_origin` as a private string. This means we cannot view or access this string from outside of our `Passport` class.

However, we have made both `passport_number` and `passport_name` public so we can interact with those two variables outside of our class. We'll look at how to do this in just a bit.

If you don't assign an access modifier to your data members or functions inside your class, C++ will assume those data members and functions are private.

C++ Class Access Modifiers

When adding data members or functions to a class, we can use one of three keywords to control their access:

Private: We can only use any class member or function listed as private within the class they belong to. Trying to access or view private data outside of their respective class results in an error.

Protected: Protected members or functions are limited in their use as well. In addition to using protected members or functions within their class, we can also access protected information from within inherited classes.

Public: We can use public members and functions anywhere else in our code.

While we don’t cover inherited classes that use the protected keyword here, below is an example of how public and private access modifiers work:

Printing From an Encapsulated Class in C++

The above example is a valid class, but it has some limitations. We are free to use or assign values to `passport_number` and `passport_name` in an instance of the `Passport` class anywhere else in our code. This also leaves these variables exposed, allowing other parts of the codebase to manipulate them in a way we had not intended.
However, the `country_of_origin` string is private and therefore inaccessible by the rest of our program. We can assign `country_of_origin` a value as we declare it, but this means we cannot change it elsewhere:

```
#include <iostream>
using namespace std;
class Passport {
private:
  string country_of_origin = "Rwanda";
public:
  int passport_number;
  string passport_name;
  void dothis() {
        cout << passport_name << "'s " << country_of_origin << " passport number is " << passport_number;
  }
};
int main() {
  Passport mypassport;
  mypassport.passport_number = 476896564;
  mypassport.passport_name = "Mugabo Jack";
  mypassport.dothis();
  return 0;
}
```

The above program runs and outputs the information in the `cout` statement:

```
MUgabo Jack's Rwanda passport
number is 476896564
```

Access encapsulated data in C++

C++ allows access to private data members and functions inside a class through the use of getters and setters. `get()` and `set()` are methods, initialized in the public section of a class, as a way to either read or write to private data.

`set()` enables us to write a value to a private variable within our class. `get()`, on the other hand, allows a function outside of the class to read private data. Using `set()` and `get()` allows us to protect data members or functions in our class while still being able to use them.
Ideally, we want to make our passport information private to prevent other users from tampering with it. With setters and getters in C++, we can still modify that data.

We create a function called `setpassport_number()` with one parameter that permits us to write a value to `passport_number`, a private variable. Similarly, we introduce `getpassport_number()` to read that value outside of our class.

Our completed class has three private variables and all the setters and getters we need to read and write to those variables. Note that setters and getters need to be public so that we can use them outside of our class. Our `Passport` class looks like this:

```
#include <iostream>
using namespace std;
class Passport {
  int passport_number;
  string passport_name;
  string country_of_origin;
public:
  void setPassport_number(int passnum) {
        passport_number = passnum;
  }
  void setPassport_name(string passname) {
        passport_name = passname;
  }
  void setCountry_of_origin(string origin) {
        country_of_origin = origin;
  }
  int getPassport_number() {
        return passport_number;
  }
  string getPassport_name() {
        return passport_name;
  }
  string getCountry_of_origin() {
        return country_of_origin;
  }
};
int main() {
  Passport mypassport;
  mypassport.setPassport_number(476896564);
  mypassport.setPassport_name("Mugabo Jack");
  mypassport.setCountry_of_origin("Rwanda");
  cout << mypassport.getPassport_name() << "'s " <<
mypassport.getCountry_of_origin() << " passport number is " <<
mypassport.getPassport_number();
  return 0;
}
```

Constructors

What would happen in the previous example if we called the member function **area** before having called **set_values**? An undetermined result, since the members **width** and **height** had never been assigned a value.

In order to avoid that, a class can include a special function called its ***constructor***, which is automatically called whenever a new object of this class is created, allowing the class to initialize member variables or allocate storage.

This constructor function is declared just like a regular member function, but with a **name** that matches the **class name** and **without any return type; not even void.**

The `Rectangle` class above can easily be improved by implementing a constructor:

```
// class constructor
#include <iostream>
using namespace std;

class Rectangle {
    int width, height;
  public:
    Rectangle (int,int);
    int area () {return (width*height);}
};

Rectangle::Rectangle (int a, int b) {
  width = a;
  height = b;
}

int main () {
  Rectangle rect (3,4);
  Rectangle rectb (5,6);
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;
}
```

```
rect area: 12
rectb area: 30
```

Constructor(next…)

The results of this example are identical to those of the previous example. But now, class `Rectangle` has no member function `set_values`, and has instead a constructor that performs a similar action: it initializes the values of `width` and `height` with the arguments passed to it.
Notice how these arguments are passed to the constructor at the moment at which the objects of this class are created:

```
Rectangle rect (3,4);
Rectangle rectb (5,6);
```

Constructors cannot be called explicitly as if they were regular member functions. They are only executed once, when a new object of that class is created.
Notice how neither the constructor prototype declaration (within the class) nor the latter constructor definition, have return values; not even `void`: Constructors never return values, they simply initialize the object.

Constructor: Overloading the constructor

Like any other function, a constructor can also be overloaded with different versions taking different parameters: with a different number of parameters and/or parameters of different types. The compiler will automatically call the one whose parameters match the arguments.
On the left , two objects of class `Rectangle` are constructed: **`rect`** and **`rectb`**. `rect` is constructed with two arguments, like in the example before.
But this example also introduces a special kind **constructor**: the *default* ***constructor***. The *default constructor* is the constructor that takes no parameters, and it is special because it is called when an object is declared but is not initialized with any arguments.
In our current example the *default constructor* is called for **`rectb`**. Note how **`rectb`** is not even constructed with an empty set of parentheses - in fact, empty parentheses cannot be used to call the default constructor:

```
// overloading class constructors
#include <iostream>
using namespace std;

class Rectangle {
    int width, height;
  public:
    Rectangle ();
    Rectangle (int,int);
    int area (void) {return (width*height);}
};

Rectangle::Rectangle () {
  width = 5;
  height = 5;
}

Rectangle::Rectangle (int a, int b) {
  width = a;
  height = b;
}

int main () {
  Rectangle rect (3,4);
  Rectangle rectb;
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;
}
```

Constructor overloading(next…)

```
Rectangle rectb;   // ok, default constructor called
Rectangle rectc(); // oops, default constructor NOT
called
```

This is because the empty set of parentheses would make of `rectc` a function declaration instead of an object declaration: It would be a function that takes no arguments and returns a value of type `Rectangle`.

Constructor: uniform initialization

The way of calling constructors by enclosing their arguments in parentheses, as shown above, is known as *functional form*. But constructors can also be called with other syntaxes:

First, constructors with a single parameter can be called using the variable initialization syntax (an equal sign followed by the argument):

```
class_name object_name = initialization_value;
```

More recently, C++ introduced the possibility of constructors to be called using *uniform initialization*, which essentially is the same as the functional form, but using braces ({}) instead of parentheses (( )):

```
class_name object_name { value, value,
value, ... }
```

Optionally, this last syntax can include an equal sign before the braces.

Here is an example with four ways to construct objects of a class whose constructor takes a single parameter:

```
// classes and uniform initialization
#include <iostream>
using namespace std;

class Circle {
    double radius;
  public:
    Circle(double r) { radius = r; }
    double circum() {return
2*radius*3.14159265;}
};

int main () {
  Circle foo (10.0);   // functional form
  Circle bar = 20.0;   // assignment init.
  Circle baz {30.0};   // uniform init.
  Circle qux = {40.0}; // POD-like

  cout << "foo's circumference: " <<
foo.circum() << '\n';
  return 0;
}
```

Constructor: uniform initialization

An advantage of uniform initialization over functional form is that, unlike parentheses, braces cannot be confused with function declarations, and thus can be used to explicitly call default constructors:

```
Rectangle rectb;   // default constructor called
Rectangle rectc(); // function declaration (default constructor NOT called)
Rectangle rectd{}; // default constructor called
```

The choice of syntax to call constructors is largely a matter of style. Most existing code currently uses functional form, and some newer style guides suggest to choose uniform initialization over the others

Member initialization in constructors

When a constructor is used to initialize other members, these other members can be initialized directly, without resorting to statements in its body. This is done by inserting, before the constructor's body, a colon (`:`) and a list of initializations for class members. For example, consider a class with the following declaration:

```
class Rectangle {
    int width,height;
  public:
    Rectangle(int,int);
    int area() {return width*height;}
};
```

The constructor for this class could be defined, as usual, as

```
Rectangle::Rectangle (int x, int y) { width=x; height=y; }
```

or

```
Rectangle::Rectangle (int x, int y) : width(x), height(y) { }
```

Or

```
Rectangle::Rectangle (int x, int y) : width(x) { height=y; }
```

Note how in this last case, the constructor does nothing else than initialize its members, hence it has an empty function body.

For members of fundamental types, it makes no difference which of the ways above the constructor is defined, because they are not initialized by default, but for member objects (those whose type is a class), if they are not initialized after the colon, they are default-constructed.

Member initialization in constructors

Default-constructing all members of a class may or may always not be convenient: in some cases, this is a waste (when the member is then reinitialized otherwise in the constructor), but in some other cases, default-construction is not even possible (when the class does not have a default constructor). In these cases, members shall be initialized in the member initialization list. For example:

```
// member initialization
#include <iostream>
using namespace std;

class Circle {
    double radius;
  public:
    Circle(double r) : radius(r) { }
    double area() {return radius*radius*3.14159265;}
};

class Cylinder {
    Circle base;
    double height;
  public:
    Cylinder(double r, double h) : base (r), height(h) {}
    double volume() {return base.area() * height;}
};

int main () {
  Cylinder foo (10,20);

  cout << "foo's volume: " << foo.volume() << '\n';
  return 0;
}
```

```
foo's volume: 6283.19
```

Pointer to class

Objects can also be pointed to by pointers: Once declared, a class becomes a valid type, so it can be used as the type pointed to by a pointer. For example:

`Rectangle * prect;` **`//is a pointer to an object of class Rectangle.`**

Similarly as with plain data structures, the members of an object can be accessed directly from a pointer by using the arrow operator (->). Here is an example with some possible combinations:

```
// pointer to classes example
#include <iostream>
using namespace std;

class Rectangle {
  int width, height;
public:
  Rectangle(int x, int y) : width(x), height(y) {}
  int area(void) { return width * height; }
};


int main() {
  Rectangle obj (3, 4);
  Rectangle * foo, * bar, * baz;
  foo = &obj;
  bar = new Rectangle (5, 6);
  baz = new Rectangle[2] { {2,5}, {3,6} };
  cout << "obj's area: " << obj.area() << '\n';
  cout << "*foo's area: " << foo->area() << '\n';
  cout << "*bar's area: " << bar->area() << '\n';
  cout << "baz[0]'s area:" << baz[0].area() << '\n';
  cout << "baz[1]'s area:" << baz[1].area() << '\n';
  delete bar;
  delete[] baz;
  return 0;
}
```

Operators to pointer and Object

expression	can be read as
*x	pointed to by x
&x	address of x
x.y	member y of object x
x->y	member y of object pointed to by x
(*x).y	member y of object pointed to by x (equivalent to the previous one)
x[0]	first object pointed to by x
x[1]	second object pointed to by x
x[n]	(n+1)th object pointed to by x

Exercises

OOP Exercises Link:
https://shorturl.at/cnA34

Pointers
https://shorturl.at/dfSTY

Build a C++ Program that Have Multiple Source Code Files

In C++ generally, we saw only a single file program but what if we have multiple source code files and we have to build a program by using all these files? Let’s see how we can build a C++ program with multiple source files without including them as header files. There are two methods to use two or more source code files in C++ as mentioned below:

1. Using header file
2. Using -gcc commands

Multiple Source Code Files

Using header files

Build a calculator and save it as calculator.h

```
namespace calculator{

int addition(int a, int b){

return a+b;
}
int multiplication(int a, int b){

return a*b;
}
```

Create test.cpp and add calculator.h using #include directive

```
#include<iostream>
#include "calculator.h"
using namespace calculator;
using namespace std;
int main(){

cout<<addition(5,10);
}
```

Compile test.cpp

Using class header file having implementation

Mixing class methods prototypes and their implementation one header file(.h) is possible but it is a **Bad Practice.**

Create rectangle.h

```
#ifndef Rectangle_H
#define Rectangle_H
class Rectangle {
    int width, height;
  public:
    Rectangle (){}
 Rectangle(int, int);
    void set_values(int w, int l){
        width=w;
       height=l;
    }
    int area (void) {return
```

Create TestRectangle.cpp

```
#include<iostream>
#include "rectangle.h"
using namespace std;
int main(){
Rectangle rect;
rect.set_values(10,20)
cout<<"The are :"<<rect.area()<<endl;
return 0;

}
```

Compile and run TestRectangle.cpp

Use gcc commands: Separate header file interface and implementation: Good practice

rectangle.h

```
#ifndef Rectangle_H
#define Rectangle_H
class Rectangle{
private:
int width;
int height;
public:
Rectangle();
Rectangle(int, int);
Void set_ values(int, int);
//Function that return a Rectangle
Rectangle rectc();
int area();
};
#endif
```

rectangle.cpp

```
#include "rectangle.h"
Rectangle::Rectangle ( ) {
  width = 5;
  height = 5;
}
Rectangle::Rectangle (int a, int b) {
  width = a;
  height = b;
}
void Rectangle::set_values (int x, int y) {
  width = x;
  height = y;
}
int Rectangle::area(){
return width*height;
}
```

Compiling

```
g++ -c rectangle.cpp -o rectangle.o
g++ -c program.cpp -o program.o
g++ rectangle.o program.o -o main
```

or

```
g++ -o main rectangle.cpp program.cpp
```

program.cpp

```
#include<iostream>
#include “rectangle.h”
int main () {
  Rectangle rect (3,4);
  Rectangle rectb;
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;
}
```

NOTE: When you see the .h file:
#ifndef CLASS_H
#define CLASS_H
/* ... Declarations etc here ... */
#endif
This is a preprocessor technique of preventing a header file from being included multiple times.